



Comp90042

Natural Language Processing

Workshop

(Week 6 | 2026s1)

Dr Jean Lee





Agenda

1. Discussion (~25 mins)

- **Transformer**
- **Point-wise Mutual Information (PMI)**
- **Singular Value Decomposition (SVD)**
- **Skip-gram & CBOW**

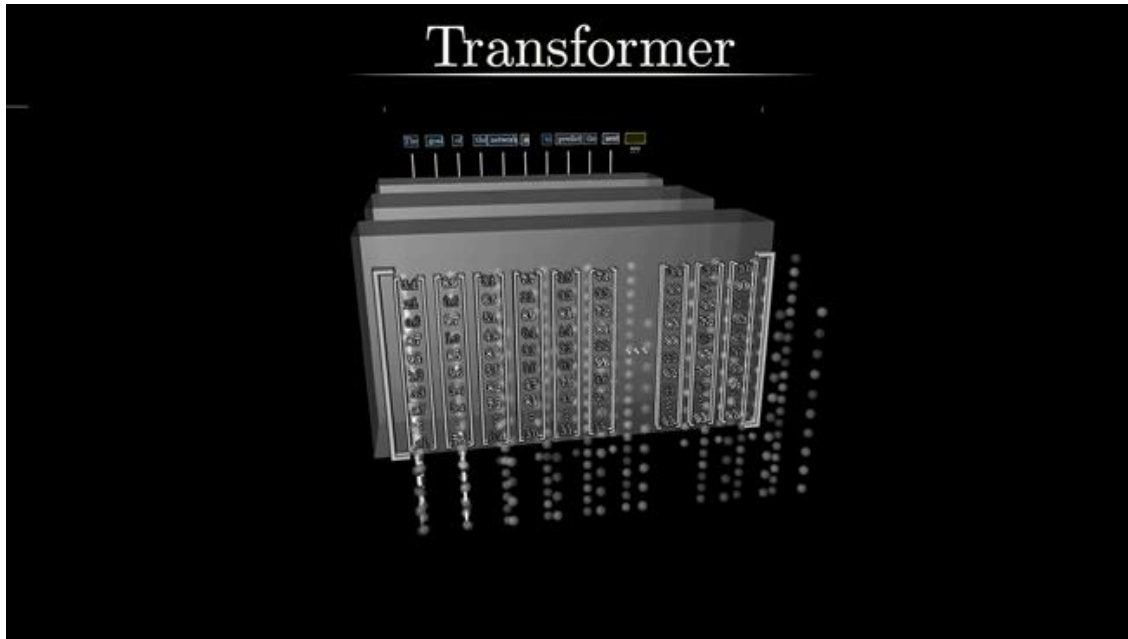
2. Programming (~30 mins)

13th April	6	 workshop-06.pdf 	08-transformer-pytorch.ipynb  09-word-embeddings.ipynb 	releasing on 20th April
------------	---	---	--	-------------------------

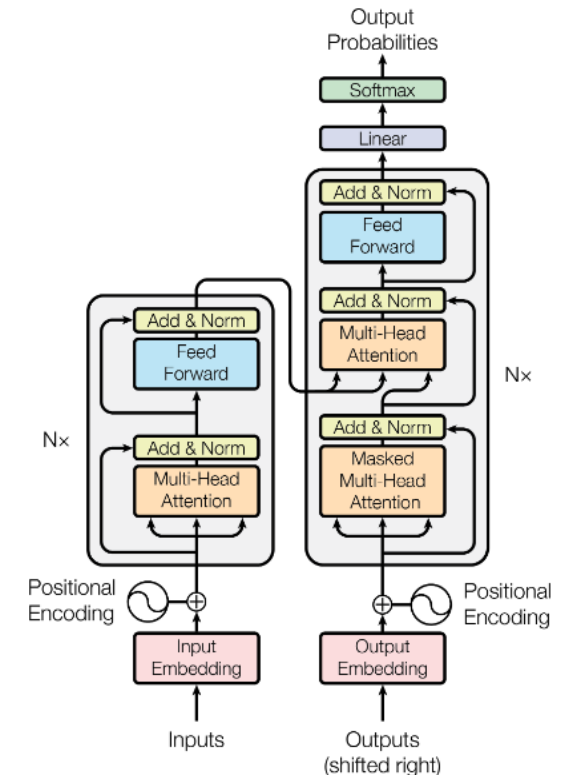
Discussion : Transformer

1. How does a **transformer** captures dependencies between words? What advantages does it have compared to RNN?

What is a Transformer?



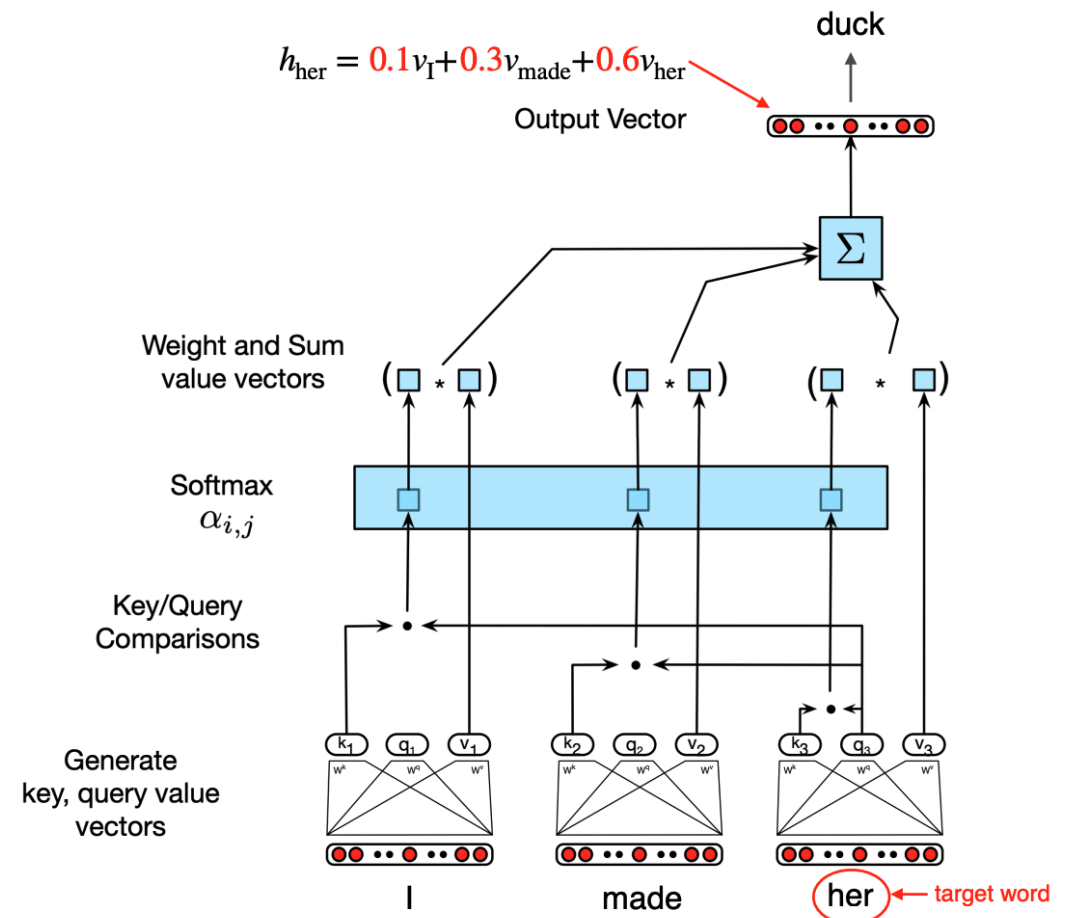
- *Encoder-Decoder Structure*
- **Attention**
- *Position-wise FF Networks*
- *Embedding & Softmax*
- *Positional Encoding*



Discussion : Transformer

1. How does a transformer captures dependencies between words? What advantages does it have compared to RNN?

- Transformer uses **attention** to capture dependencies between words.
- For each **target word (Query vector)** in a sentence, transformer **attends to every** other words (**Key vectors**) in the sentence to create its **contextual** embeddings.
- Final representation of target word = weighted sum of **value vectors of context words and target word**.



Discussion : Transformer

1. How does a **transformer** captures dependencies between words? What advantages does it have compared to RNN?

What is Self-Attention?

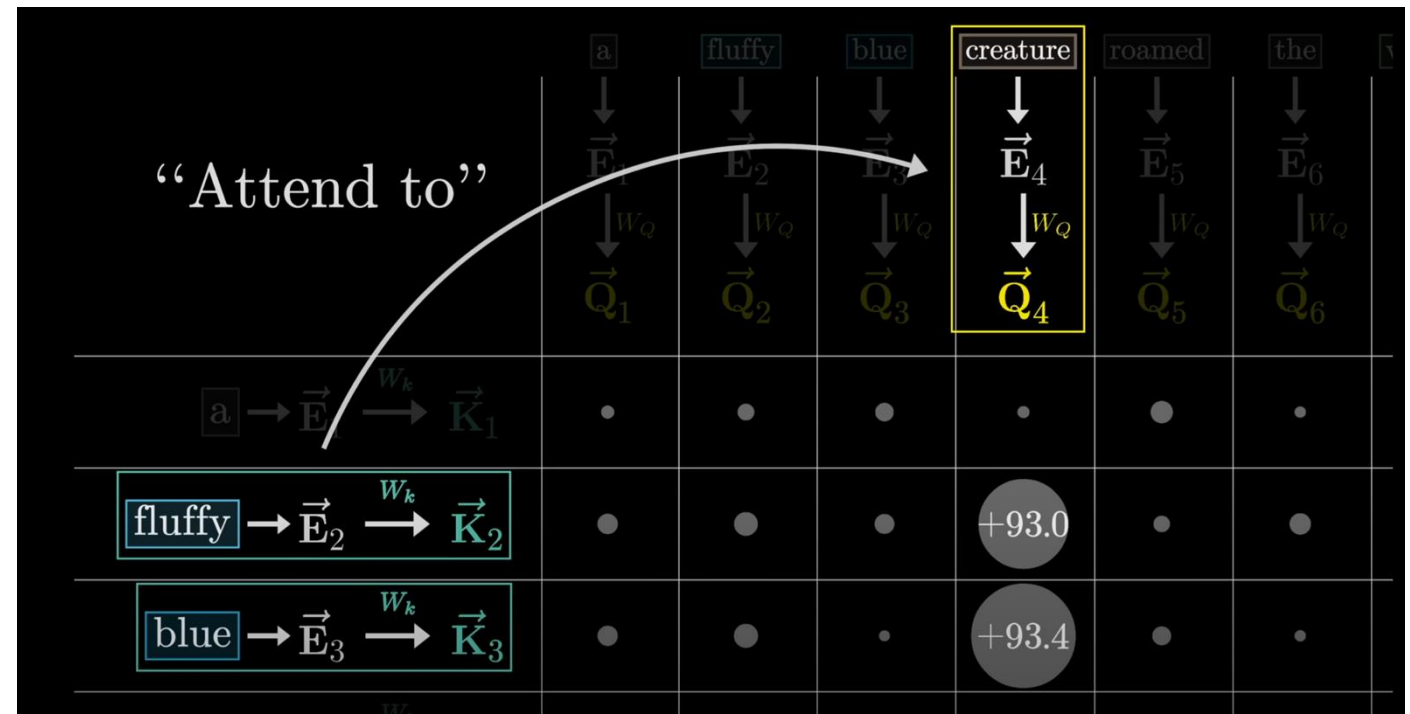
SelfAttention(q, K, V)

$$= \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} \times v_i \quad h_{\text{her}} \in \mathbb{R}^{1 \times d_v}$$

← query and key comparisons

← softmax

- As the **computation** of the contextual embeddings of a target word is **independent to other target words**, we can **parallelise** the computation.



Attention in transformers, step-by-step :

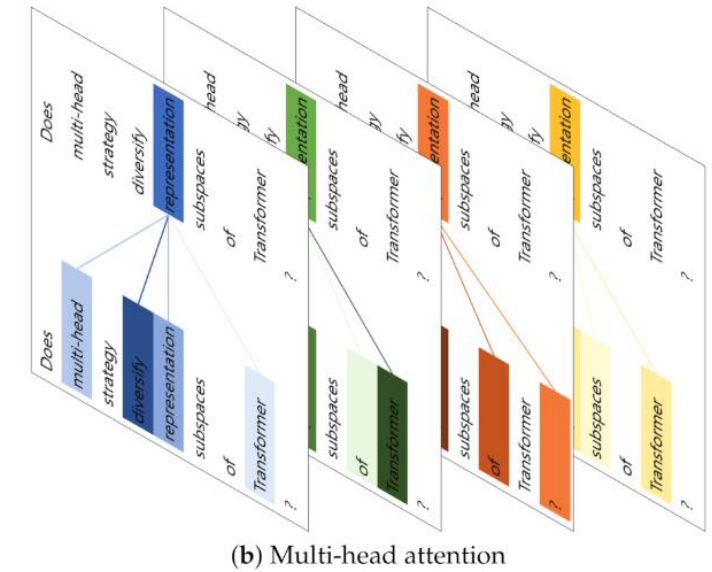
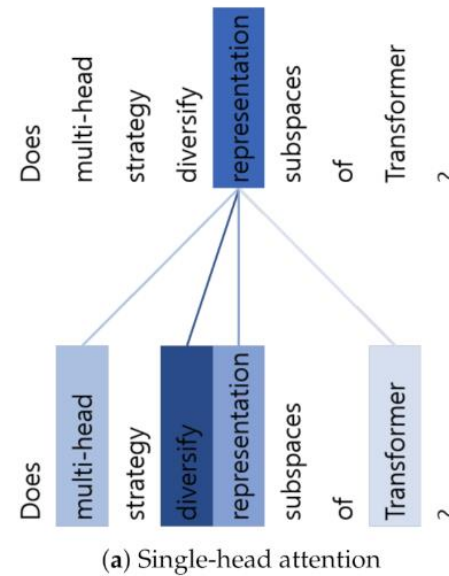
<https://youtu.be/eMlx5fFNoYc?si=MwjiXS1JrsUnpyoC>

Discussion : Transformer

1. How does a **transformer** captures dependencies between words? What advantages does it have compared to RNN?

- **Parallel Processing**

- Unlike RNNs, transformers process all tokens in parallel.
- **scale to very large** data (and faster)
- RNN rely on **sequential processing**
- the contextual embedding of the current target word **cannot be computed** until we have processed the previous word.





Point-wise Mutual Information (PMI)

2. For the following term co-occurrence matrix (suitably interpreted):

	cup	not (cup)
world	55	225
not (world)	315	1405

(a) Find the Point-wise Mutual Information (PMI) between these two terms in this collection.

- **PMI** represent how often two events co-occur

$$\text{PMI}(x, y) = \log_2 \frac{P(x, y)}{P(x)P(y)}$$

$$P(x, y) = \frac{\text{count}(x, y)}{\Sigma}$$

$$P(x) = \frac{\Sigma_x}{\Sigma}$$

$$P(y) = \frac{\Sigma_y}{\Sigma}$$

Σ (total # of instance)	55+225+315+1405 = 2000
P(world)	(55 + 225) / 2000 = 0.14
P(cup)	(55 + 315) / 2000 = 0.185
P(world, cup)	55 / 2000 = 0.0275
PMI (world, cup)	$\log_2(0.0275 / (0.14 * 0.185))$ ≈ 0.0865

Point-wise Mutual Information (PMI)

(b) What does the value from (a) tell us about the **distributional properties** between these two terms?

- *PMI represent how often two events co-occur*

PMI (world, cup)

$\log_2(0.0275 / (0.14 * 0.185))$

≈ 0.0865

- This value is **slightly positive**
- the two events occur together (in documents) **slightly more commonly** than would occur purely by chance.
- There is some possibility that world and cup occurring together is somehow meaningful for documents in this collection. (e.g. **World Cup**)

Singular Value Decomposition (SVD)

3. In the 09-word-embeddings iPython notebook, a term-document matrix is built to learn word vectors.

(a) What is the Singular Value Decomposition (SVD) method used for here? Why is this helpful?

SVD : Dimensionality reduction

- sparse -> dense
- More practical (less features)
- Remove noise (less overfitting)

It is **always** possible to decompose a real matrix A into $A = U \Sigma V^T$, where

- U, Σ, V : **unique**
- U, V : **column orthonormal**
 - $U^T U = I; V^T V = I$ (I : identity matrix)
 - (Columns are orthogonal unit vectors)
- Σ : **diagonal**
 - Entries (**singular values**) are **positive**, and sorted in decreasing order ($\sigma_1 \geq \sigma_2 \geq \dots \geq 0$)

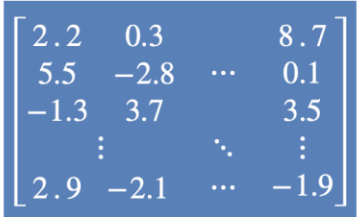
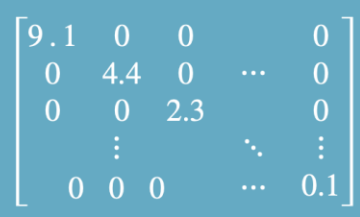
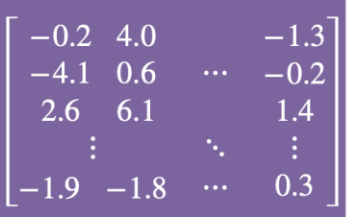
$$A = U \Sigma V^T$$

$|D|$
 $|V|$  A (term-document matrix)

$\Downarrow$

U (new term matrix) Σ (singular values) V^T (new document matrix)

m m $|D|$

$|V|$  m  m  m

$m = \text{Rank}(A)$

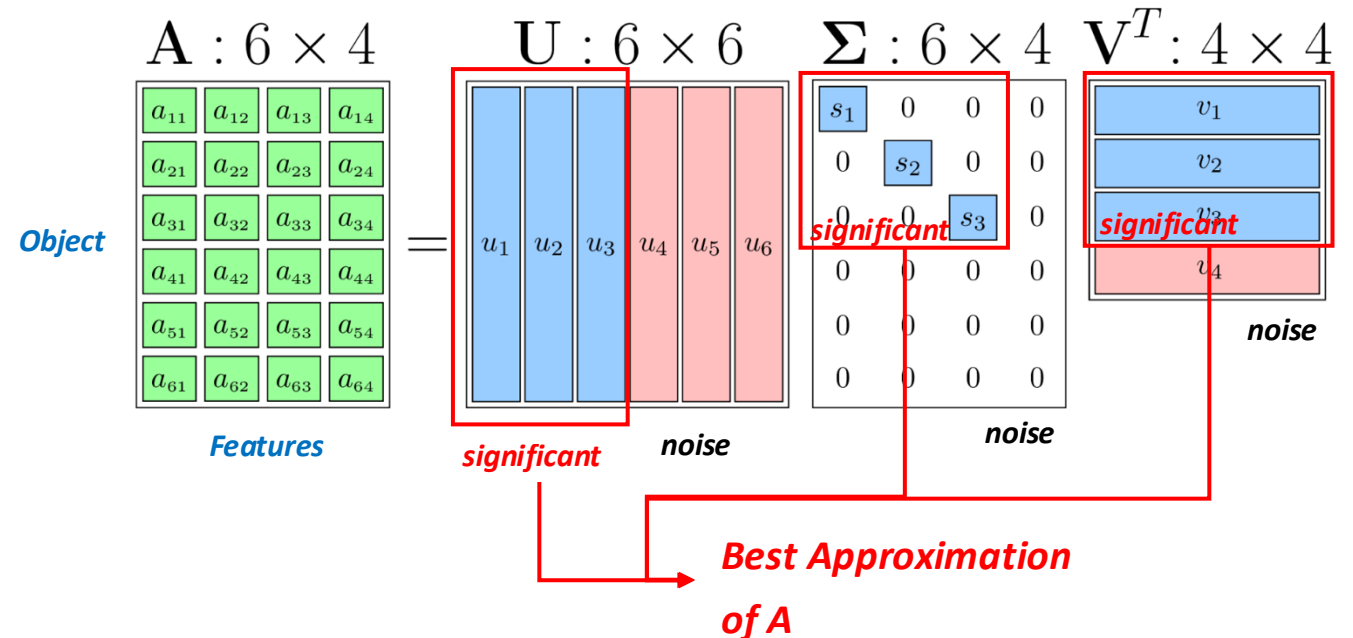
Singular Value Decomposition (SVD)

3. In the 09-word-embeddings iPython notebook, a term-document matrix is built to learn word vectors.

(a) What is the Singular Value Decomposition (SVD) method used for here? Why is this helpful?

SVD

- **A** : Input data matrix
- **U** : Left singular vectors
- **Σ** : Singular values
 - Diagonal matrix of $r \times r$
 - r : rank of the matrix A
 - strength of each features
- **V** : Right singular vectors



SVD Gives the Best Low Rank Approximation

<https://youtu.be/c7e-D2tmRE0?si=AxGQm3rPKsaQV7Ly>

Singular Value Decomposition (SVD)

3. In the 09-word-embeddings iPython notebook, a term-document matrix is built to learn word vectors.

(a) What is the Singular Value Decomposition (SVD) method used for here? Why is this helpful?

- to identify the most important characteristics of words.
- By throwing away the less important characteristics,
 - have a smaller representation of the words
 - save time and storage

```
1 from sklearn.decomposition import TruncatedSVD
2 from scipy.sparse import csr_matrix
3
4 #dimension of brown_matrix_tfidf = num_documents x num_vocab
5 #dimension of brown_matrix_tfidf_transposed = num_vocab x num_documents
6
7 brown_matrix_tfidf_transposed = csr_matrix(brown_matrix_tfidf).transpose()
8
9 svd = TruncatedSVD(n_components=10)
10 brown_matrix_lowrank = svd.fit_transform(brown_matrix_tfidf_transposed)
11
12 print(brown_matrix_lowrank.shape)
13 print(brown_matrix_lowrank)
```

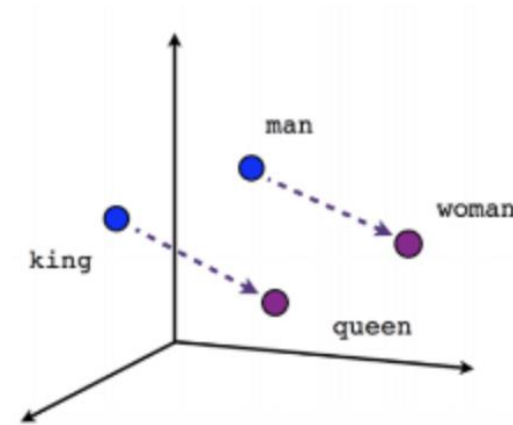
```
→ (49815, 10)
[[ 1.46529922e+02 -1.56578300e+02  3.77295895e+01 ...  1.87574330e+01
   5.17826940e+00 -1.32357467e+01]
 [ 6.10797743e-01  6.77336542e-01 -2.04392054e-01 ... -1.02796238e+00
  -1.14385161e-01 -1.12871217e+00]
 [ 1.00411586e+00  1.99456979e-01 -1.25054329e-01 ...  1.14578446e+00
  -4.14250674e-01 -1.68706426e-01]
 ...
```

Word Embedding & Distributional Similarity

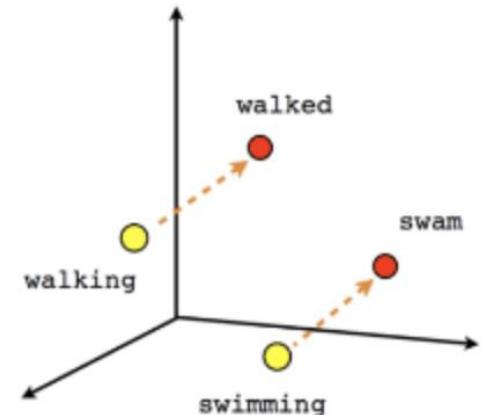
4. What is a **word embedding** and how does it relate to **distributional similarity**?
- (a) What is the difference between a **skip-gram** model and a **CBOW** model?
 - (b) How are the above models trained?

Word Embedding

- Representing words by their context in a **vector space**
- Broadly same as **Distributional Similarity**
- row corresponding to the word in the **relevant (target/context) matrix** is known as the “embedding”



Male-Female



Verb tense

Skip-gram & Continuous Bag of Words (CBOW)

4. What is a **word embedding** and how does it relate to **distributional similarity**?

- (a) What is the difference between a **skip-gram** model and a **CBOW** model?
- (b) How are the above models trained?

*Skip-gram : predict **surrounding words** of target word*



*CBOW: predict **target word** using surrounding words*

Skip-gram & Continuous Bag of Words (CBOW)

4. What is a **word embedding** and how does it relate to **distributional similarity**?

(a) What is the difference between a **skip-gram** model and a **CBOW** model?

(b) How are the above models trained?

- Taking the **dot product** of the relevant vectors (or average of vectors, in the case of CBOW)
- and then **marginalising**. (normalisation over $|V|$)

$$P(\text{life} | \text{rests}) = \frac{\exp(W_{\text{rests}} \cdot C_{\text{life}})}{\sum_{u \in V} \exp(W_{\text{rests}} \cdot C_u)}$$

word embedding of rests (points to W_{rests})
word embedding of life (points to C_{life})
dot product (points to the dot in $W_{\text{rests}} \cdot C_u$)

28

$$P(+|t, c) = \frac{1}{1 + e^{-t \cdot c}}$$

← **maximise** similarity between target word and real context words

$$P(-|t, c) = 1 - \frac{1}{1 + e^{-t \cdot c}}$$

← **minimise** similarity between target word and non-context words

To improve efficiency, we can also use a **negative sampling**.

- negative sampling aims at **maximising** the similarity of the words in the same context and **minimising** it when they occur in different contexts



Programming!

Programming

1. In 08-transformer iPython notebook, we'll now use a transformer to do sentiment analysis:
 - (a) If the number of model parameters is similar, does the transformer model perform better than the feedforward model we built in 07-deep-learning-pytorch?
 - (b) How much slower is the transformer model compared to the feedforward model?
 - (c) Does increasing the number of model parameters for the transformer model improve performance?
2. Use the 09-word-embeddings iPython notebook to find some interesting collocations, using PMI.
 - (a) Write a wrapper function which finds the 10 collocations with the greatest PMI, amongst all of bi-grams in the collection. (Note that you might want to be careful about your strategy for doing this in a very large collection!)
 - (b) NLTK has a built-in method `collocations()` (of a `Text` object). Does it produce the same collocations as PMI? Why do you think this is the case?